

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут телекомунікацій, радіоелектроніки та електронної техніки
кафедра «Радіоелектронні пристрої та системи»



Звіт з лабораторної роботи №10
з дисципліни «Об'єктно-орієнтоване програмування, частина 2»

Підготував:
ст. групи ІХ-21
Пасько Максим Павлович

Прийняла:
к.т.н., доц. каф. ПАСІ
Плесканка М.В.

Львів 2026р.

Лабораторна робота №10 Модульне тестування об'єктно-орієнтованих програм у Python

Мета: Навчитися створювати модульні тести для класів і функцій у Python з використанням бібліотеки unittest, у тому числі з використанням параметризованих тестів і мок-об'єктів.

Хід виконання роботи

Завдання на самостійну роботу

1. Тестування виконання арифметичних операцій

Створіть клас MathTool, який реалізує основні арифметичні операції:

- складання чисел
- різниця між двома числами
- множення
- ділення з перевіркою на поділ на нуль

Реалізуйте модульні тести до кожної з цих операцій. Перевірте поведінку методу ділення у випадку, коли другий аргумент дорівнює нулю.

```
import unittest
```

```
class MathTool:
```

```
    def add(self, a, b):  
        return a + b
```

```
    def subtract(self, a, b):  
        return a - b
```

```
    def multiply(self, a, b):  
        return a * b
```

```
    def divide(self, a, b):  
        if b == 0:  
            raise ValueError("Ділення на нуль")  
        return a / b
```

```
class TestMathTool(unittest.TestCase):
```

```
    def setUp(self):  
        self.math = MathTool()
```

```
    def test_add(self):  
        self.assertEqual(self.math.add(5, 3), 8)
```

```

def test_subtract(self):
    self.assertEqual(self.math.subtract(10, 4), 6)

def test_multiply(self):
    self.assertEqual(self.math.multiply(3, 5), 15)

def test_divide(self):
    self.assertEqual(self.math.divide(10, 2), 5)

def test_divide_by_zero(self):
    with self.assertRaises(ValueError):
        self.math.divide(10, 0)

if __name__ == "__main__":
    unittest.main()

```

Виведе:

Ran 5 test in 0.002s

OK

Process finished with exit code 0

2. Тестування класу LibraryItem

Створіть клас LibraryItem, який має поля: title, author, year. Додайте метод details(), який повертає рядок із повною інформацією про об'єкт.

Напишіть юніт-тест, який перевіряє, чи коректно метод details() формує інформацію для різних прикладів.

```

import unittest

class TestLibraryItem(unittest.TestCase):

    def test_details(self):
        book = LibraryItem("Кобзар", "Тарас Шевченко", 1840)
        self.assertEqual(
            book.details(),
            "Назва: Кобзар, Автор: Тарас Шевченко, Рік: 1840"
        )

    def test_details_second_example(self):
        book = LibraryItem("1984", "George Orwell", 1949)
        self.assertEqual(
            book.details(),
            "Назва: 1984, Автор: George Orwell, Рік: 1949"
        )

```

```
class LibraryItem:
```

```
    def __init__(self, title, author, year):
        self.title = title
        self.author = author
        self.year = year

    def details(self):
        return f'Назва: {self.title}, Автор: {self.author}, Рік: {self.year}'
```

Виведе:

Ran 2 test in 0.002s

OK

Process finished with exit code 0

3. Тестування взаємодії класів з використанням mock

Створіть два класи: NotificationService (має метод send) та UserManager, який використовує NotificationService для надсилання повідомлень. В класі UserManager реалізуйте метод notify_user.

Створіть мок-об'єкт для NotificationService і протестуйте, що notify_user коректно викликає метод send із заданими параметрами.

```
import unittest
from unittest.mock import MagicMock

class TestUserManager(unittest.TestCase):

    def test_notify_user(self):
        mock_service = MagicMock()

        manager = UserManager(mock_service)

        manager.notify_user("Maxim", "Вітаємо!")

        mock_service.send.assert_called_once_with("Maxim", "Вітаємо!")

class NotificationService:

    def send(self, user, message):
        print(f'Повідомлення для {user}: {message}')

class UserManager:

    def __init__(self, notification_service):
```

```
self.notification_service = notification_service
```

```
def notify_user(self, user, message):  
    self.notification_service.send(user, message)
```

Виведе:

Ran 1 test in 0.003s

OK

Process finished with exit code 0

4. Параметризовані тести для парності числа

Опишіть функцію `check_even(number)`, яка повертає `True`, якщо число парне, і `False` інакше.

Створіть набір параметризованих тестів із використанням

`@parameterized.expand`, щоб перевірити функцію на різних прикладах: позитивні числа, від'ємні, нуль, непарні значення.

```
import unittest  
from parameterized import parameterized  
  
class TestCheckEven(unittest.TestCase):  
  
    @parameterized.expand([  
        (2, True),  
        (4, True),  
        (0, True),  
        (-2, True),  
        (3, False),  
        (7, False),  
        (-5, False)  
    ])  
    def test_check_even(self, number, expected):  
        self.assertEqual(check_even(number), expected)  
  
def check_even(number):  
    return number % 2 == 0
```

Виведе:

Ran 7 test in 0.003s

OK

Process finished with exit code 0

Питання для самоперевірки

1. Що таке модульне тестування, і для чого воно використовується у розробці Python-програм?

Модульне тестування – перевірка окремих функцій або класів на правильність. Використовується для швидкого виявлення помилок та підвищення надійності коду.

2. Які основні методи `unittest.TestCase` використовуються для перевірки результатів тестів? Наведіть приклади.

Основні методи `unittest.TestCase`:

`assertEqual(a, b)` – перевірка рівності

`assertNotEqual(a, b)` – перевірка нерівності

`assertTrue(x)` / `assertFalse(x)` – перевірка істинності/хибності

`assertRaises(Exception, func, *args)` – перевірка виклику винятку

3. Як працюють методи `setUp()` і `tearDown()` у класі тестів Python? Наведіть приклад використання.

`setUp()` і `tearDown()` – виконуються перед і після кожного тесту для підготовки.

```
def setUp(self): self.obj = MyClass()
```

```
def tearDown(self): del self.obj
```

4. Що таке мок-об'єкт (Mock) і в яких випадках його доцільно використовувати?

Мок-об'єкт (Mock) – штучний об'єкт для імітації поведінки зовнішніх сервісів, щоб тестувати виклики методів без реальних дій.

5. Як у Python реалізуються параметризовані тести і яка їхня користь? Наведіть приклад.

Параметризовані тести – дозволяють запускати один тест з різними даними.

```
@parameterized.expand([(2, True), (3, False)])
```

```
def test_even(self, n, expected): self.assertEqual(is_even(n), expected)
```